

Monitoring Setup for Book Recommendation System

1. Objective

Monitor the health, performance, and resource usage of the microservices to ensure:

- The frontend is responsive.
 - Each microservice (book catalog, rating review, recommendation, user service) is functioning correctly.
 - API calls between microservices are timely and reliable.
 - Early detection of failures or bottlenecks.
-

2. Prometheus: Metrics Collection

- **Deployment:** Prometheus runs as a container (or pod in Kubernetes) and scrapes metrics from all services.
- **Targets:** Each microservice exposes metrics via an HTTP endpoint (e.g., `/metrics`). Metrics include:
 - **CPU and Memory Usage:** How much resources each microservice consumes.
 - **Request Count & Latency:** Number of requests received, response times, error rates.
 - **Custom Metrics:**
 - Number of books retrieved from catalog.
 - Number of recommendations generated.
 - Number of user logins or ratings submitted.

3. Grafana: Visualization

- **Deployment:** Grafana runs as a separate container and connects to Prometheus as a data source.
- **Dashboards:** Create dashboards for each microservice:
 - **Frontend Dashboard:**
 - Requests per second
 - Latency and error rates
 - **Book Catalog Dashboard:**
 - Total books requested
 - Response time
 - Errors
 - **Rating & Review Dashboard:**
 - Number of ratings submitted
 - Average rating latency
 - Failed API calls
 - **Recommendation Dashboard:**
 - Recommendations generated per minute
 - Latency for recommendation API
 - **User Service Dashboard:**
 - Active users
 - Login attempts & failures
- **Alerts:** Configure Grafana alerts (via Prometheus) for:
 - High CPU or memory usage.
 - High API response time.
 - Service downtime.

4. Workflow in Your Microservice System

1. Each microservice exposes Prometheus metrics endpoint (`/metrics`).
2. Prometheus scrapes these endpoints periodically.
3. Collected metrics are stored in Prometheus time-series database.
4. Grafana reads metrics from Prometheus and visualizes them.
5. Alerts are triggered if thresholds are breached (e.g., frontend latency > 500ms, any service down).

5. Benefits

- Helps identify bottlenecks or failing microservices.
- Provides historical performance data.
- Enables proactive scaling decisions (autoscaling based on metrics).
- Easy visualization for stakeholders or demo purposes.